



**270 Billion USD in wildfire damages
in the US alone in 2024.**



Classification of Wildfire Size

Final report presentation

Big Data Methods

07 May 2025

1. Introduction
2. Data
3. Models
4. Conclusion

Data

2

Original dataset

- 1.88 million georeferenced US wildfire records from 1992–2015 (Public SQLite Database on Kaggle)
 - Generated to support National Fire Program Analysis (FPA) system
- **Problem: no data on meteorological or topographical features**

Fire Program Analysis (FPA)

Relevant included variables
Fire Size Class
Fire Size
Latitude
Longitude
Date
Cause

Improved Dataset

- Author merged 50,000 largest fire records with...
 - Historical weather data
 - Historical vegetation data
 - Metric representing remoteness

Fire Program Analysis (FPA)



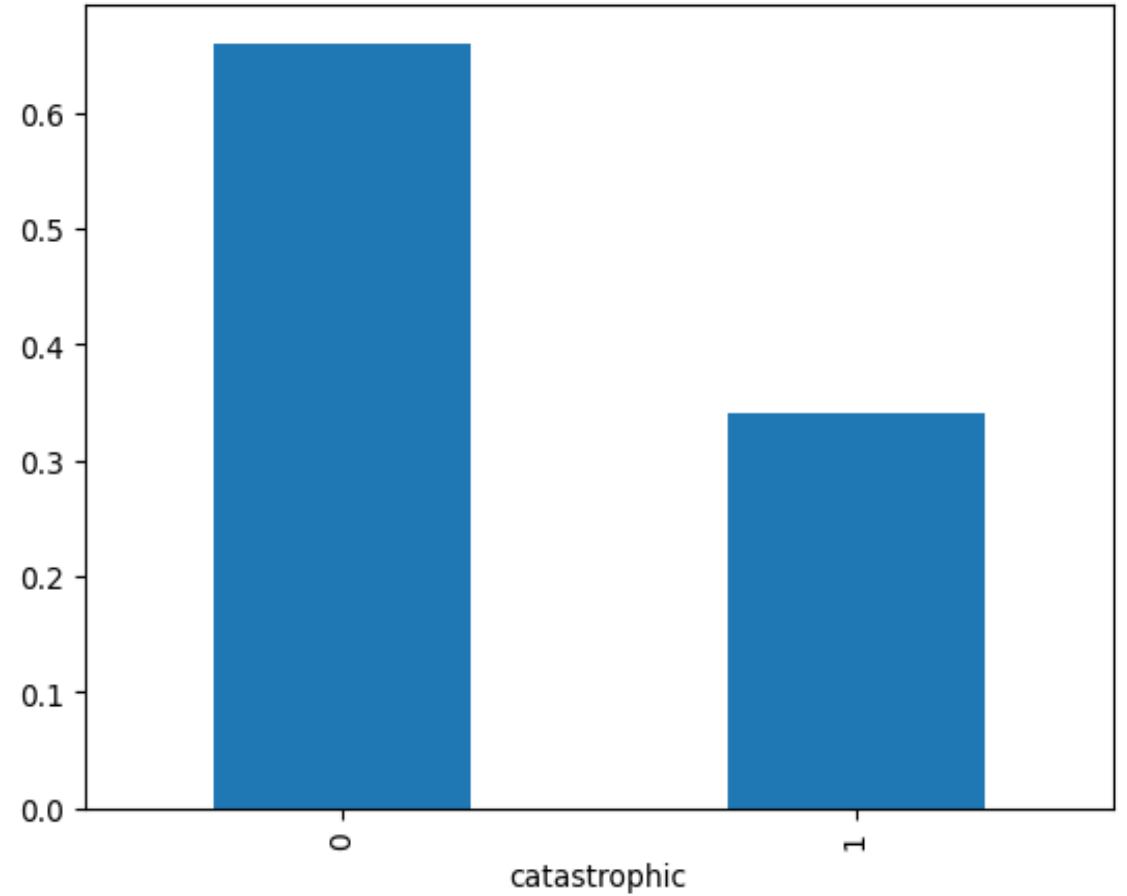
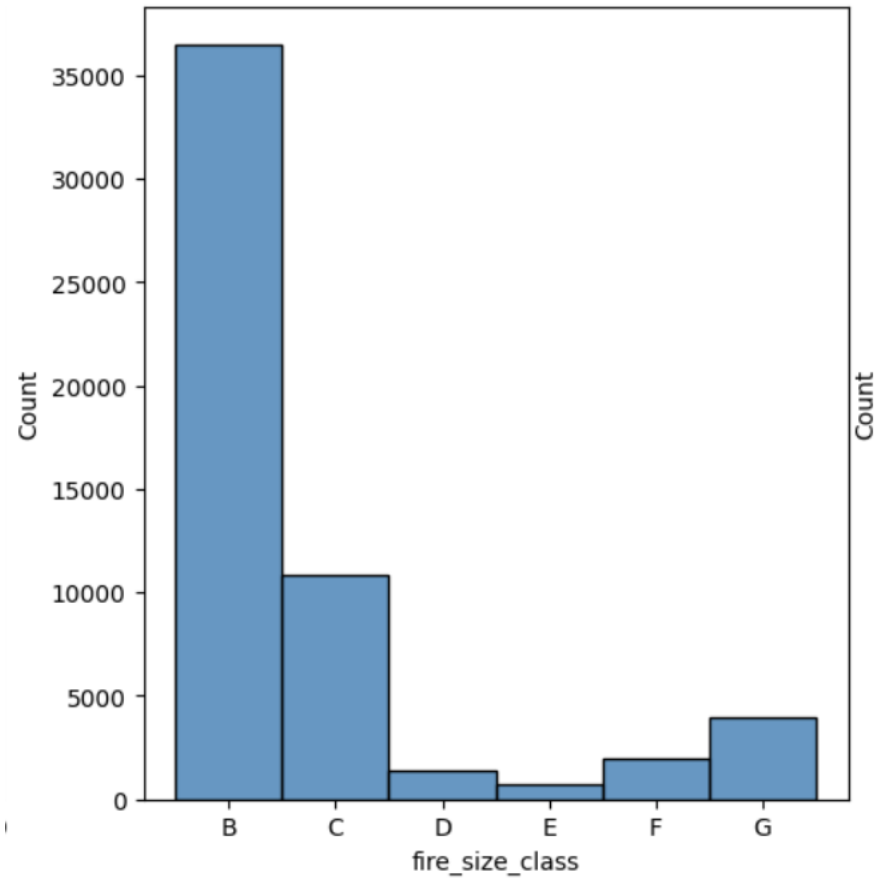
Three distinct global estimates of historical land-cover change and land-use conversions for over 200 years

Prasanth MEIYAPPAN, Atul K. JAIN (✉)

Department of Atmospheric Sciences, University of Illinois, Urbana, IL 61801, USA

Relevant included variables
Fire Size Class (B-G)
Fire Size
Latitude
Longitude
Date
Cause
Temperature Average (Last 7/15/30 days)
Wind Speed Average (Last 7/15/30 days)
Humidity Average (Last 7/15/30 days)
Precipitation Average (Last 7/15/30 days)
Vegetation Type (28 different labels)
Remoteness

Reframing the problem.



Cleaning

55,367 observations



Drop or Replace Missing Data



37,207 observations

Iterate over Temperature, Wind, and Humidity Triplets for each row...

Drop Row!

Wind_pre_30	Wind_pre_15	Wind_pre_7
-1	-1	-1
0	0	0
8	9	15
15	9	0
0	6	8
7	0	5
6	0	0
0	9	0
0	0	4

Replace!

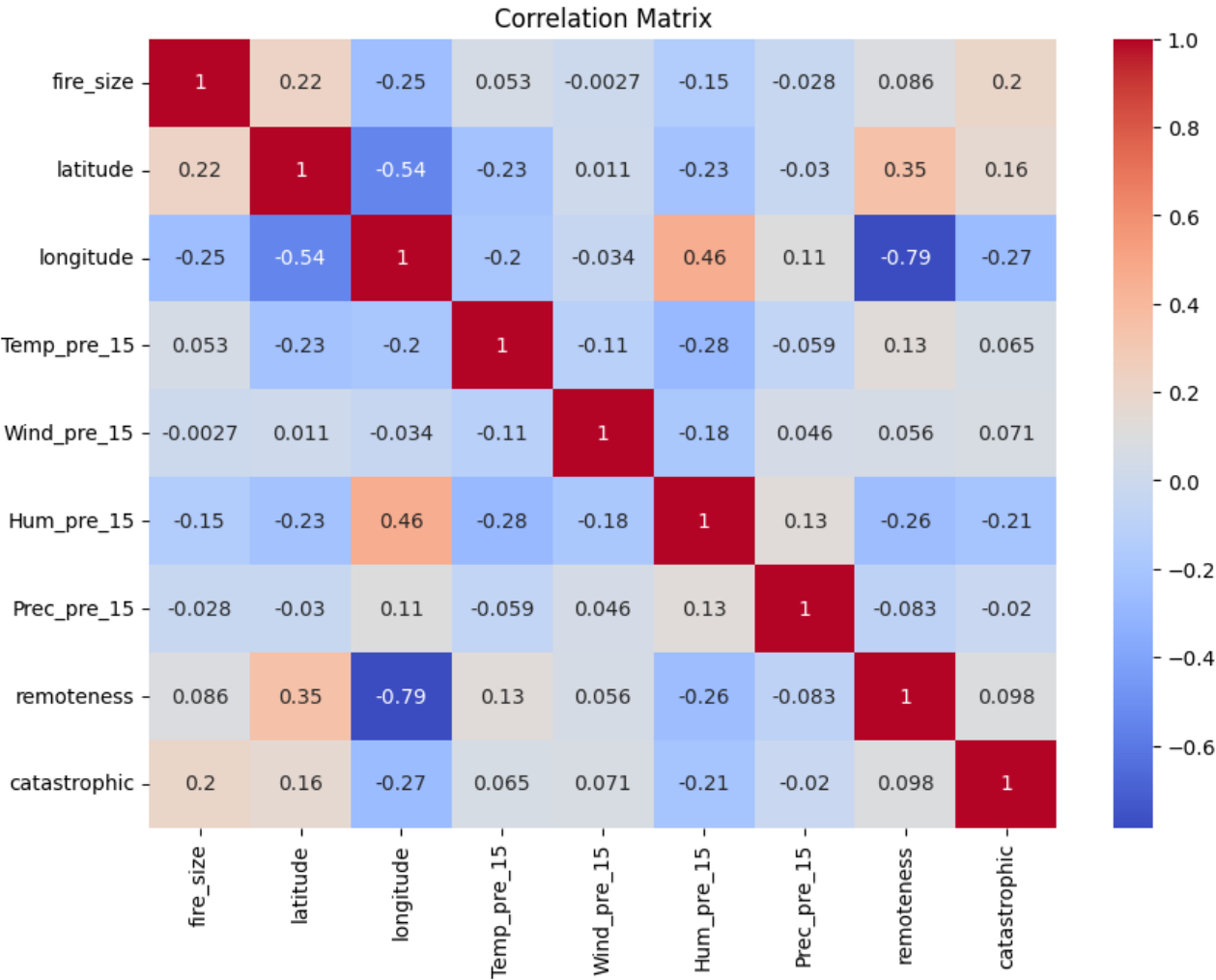
Transformation – One-Hot Encoding

Vegetation		Desert	Polar	...
Desert		1	0	...
Polar	→	0	1	...
Forest		0	0	...

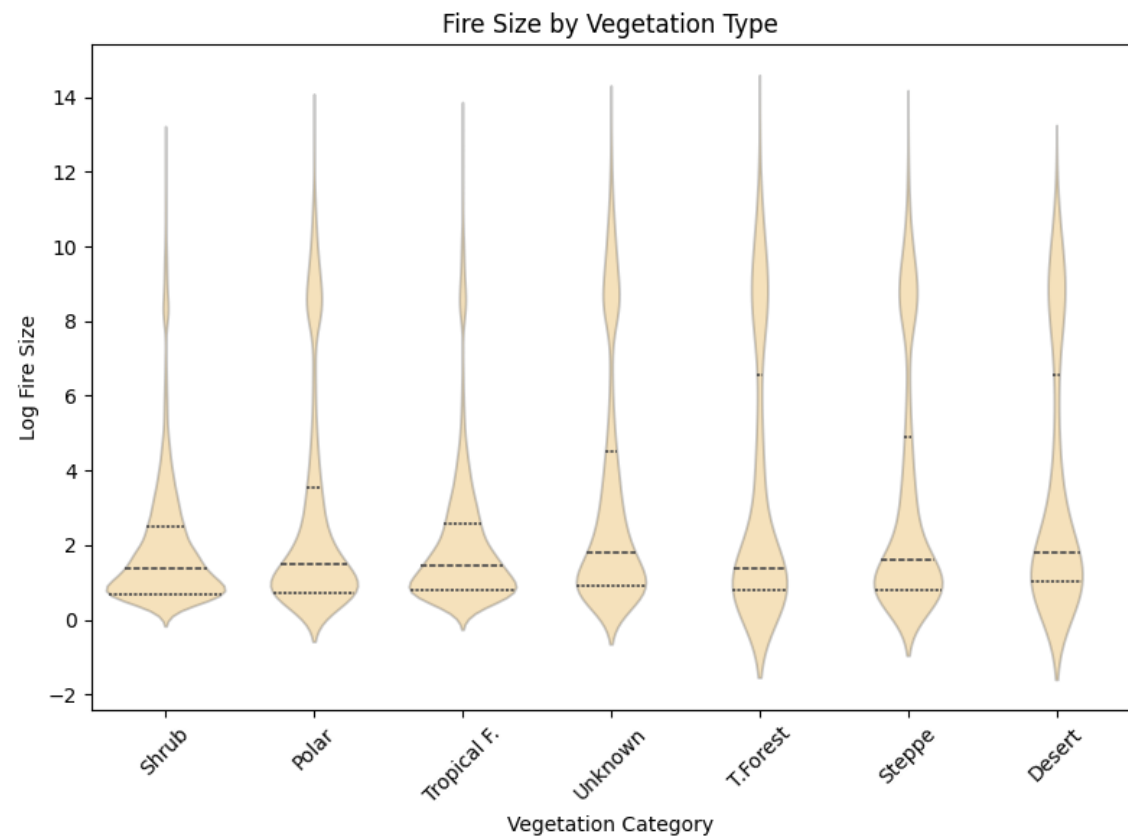
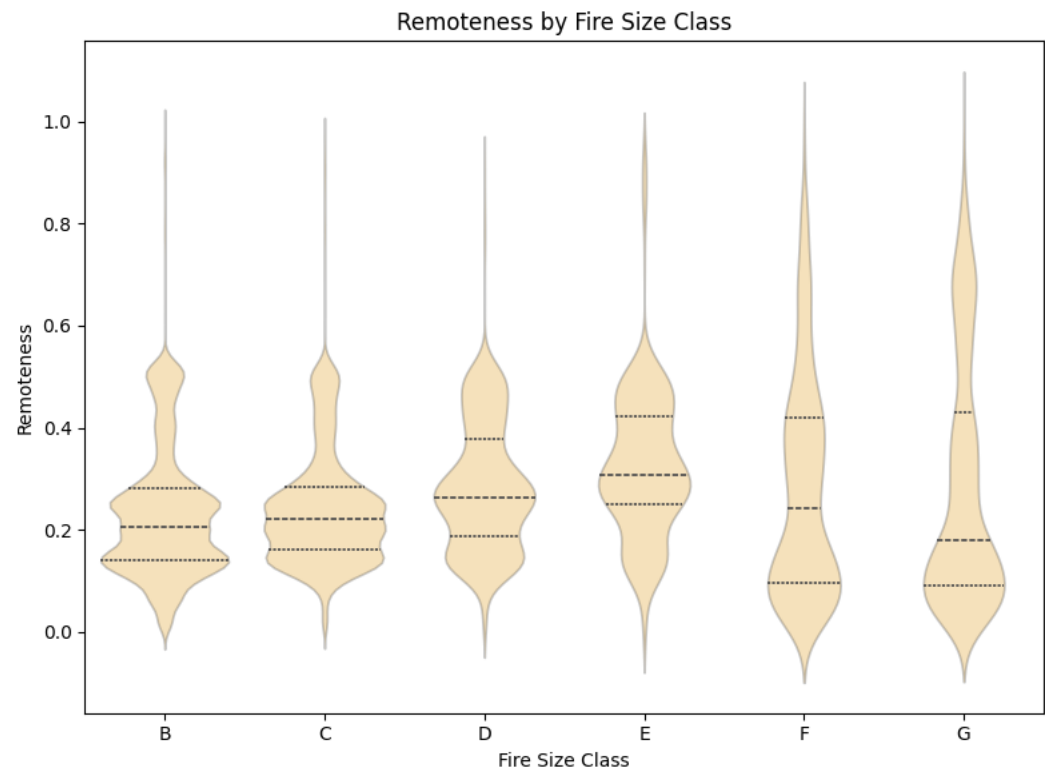
Transformation – Mapping Month to Unit Circle

Discovery Month	$\theta = \frac{2\pi(m-1)}{12}$	$\sin \theta$	$\cos \theta$	...
January		0	1	...
June		0.5	-0.866	...
December		-0.5	0.866	...

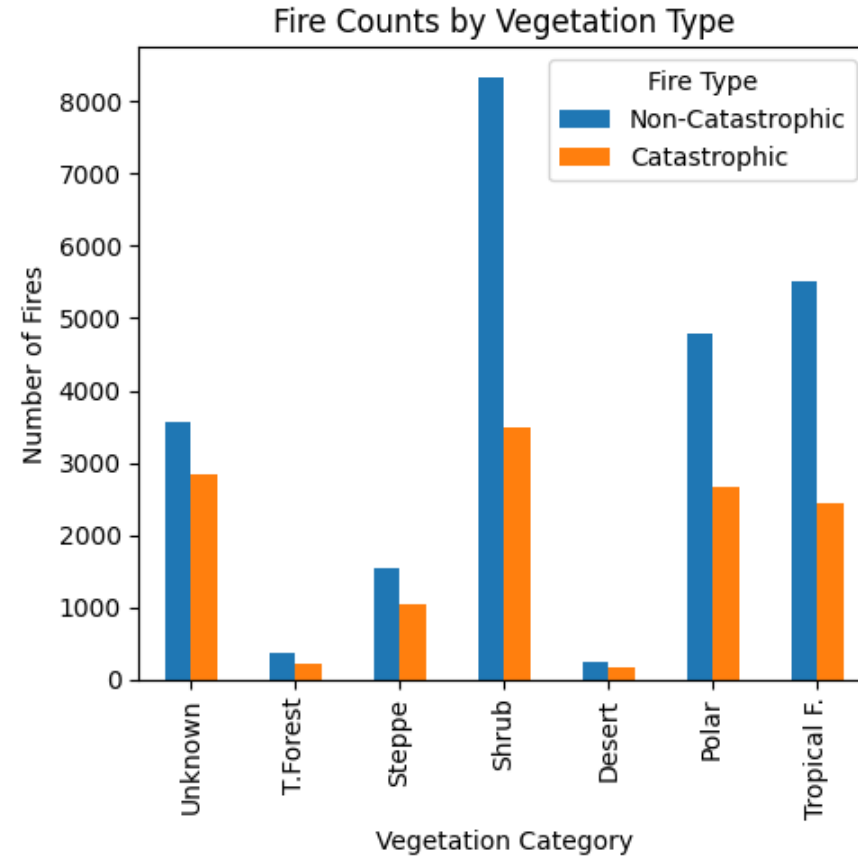
Descriptive Analysis



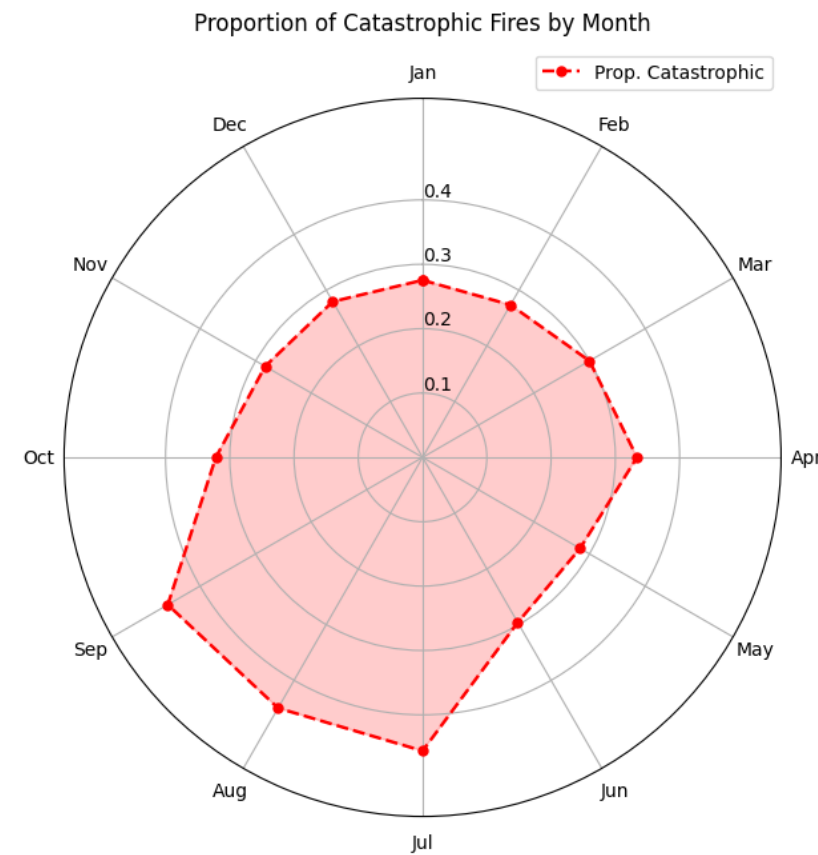
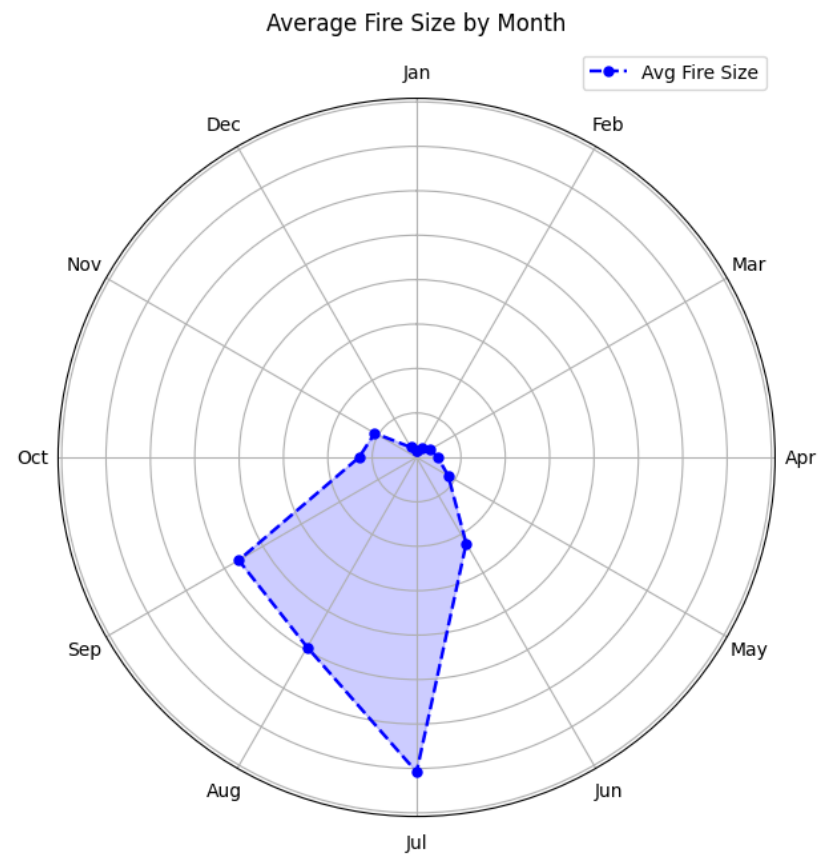
Descriptive Analysis



Descriptive Analysis



Descriptive Analysis



Models

3

Logistic Regression

3.1

Logistic Regression

- Classification Algorithm used for binary classification
- Models the probability that a given input belongs a particular class (Catastrophic or not).
- Use it as our Baseline Model
- Use GridSeachCV to tune the hyperparameters
- Scoring metric is the recall rate on the minority class (Catastrophic)
 - Focus on the most important class

Parameter	Values Tested	Best Values
logreg__C	{0.001, 0.01, 0.1, 1, 10, 100}	0.001
logreg__penalty	{L1, L2}	L1
logreg__solver	{liblinear, saga}	liblinear
logreg__max_iter	{500, 1000}	500

Table 2: Parameter Grid for Logistic Regression (GridSearchCV)

Performance

- Overall accuracy: 65.8%
- Recall score catastrophic fires: 54%
 - trade-off: accepting some false positives to better catch true catastrophic fires
- Low humidity increases the likelihood of the fire being catastrophic
- Coefficient interpretation:
 - Longitude: $e^{-0.361} \approx 0.697$
 - The odds of a fire being catastrophic are multiplied by 0.697 for each degree increase in longitude.
 - If a fire occurs 1 degree further east, it is about 30 pct. less likely (odds-wise) to be catastrophic
 - Model is interpretable and regularized, but performance could likely be improved with non-linear classifiers.

Metric / Feature	Overall	Catastrophic (Class 1)	Other (Class 0)
Test Accuracy	0.658		
Recall		0.54	0.72
Precision		0.51	0.75
F1 Score		0.52	0.73
CV Recall		0.531	
Key Non-Zero Coefficients			
Longitude	−0.361		
Humidity_pre_15	−0.096		
Humidity_pre_7	−0.020		

Table 3: Summary of Logistic Regression Results (Catastrophic = 1)

Random Forest

3.2

Introduction and hyperparameter tuning

- Method is powerful and flexible: works well for classification problems
- Improved compared to other methods: only consider a random subset of predictors at each split → decorrelates the trees
 - More stable and reliable predictions
- Use RandomizedSearchCV to explore a wide range of parameter combinations efficiently
 - Randomly selects a number of parameter combinations (20), trains and evaluates a model for each using 5-fold cross-validation, and returns the best-performing set.
- Use class weighting and recall score of catastrophic fires
- Trade off: Run time vs. Performance
 - Overfitting?

Parameter	Values Tested	Best Value
n_estimators	Random integers between 100 and 300	120
max_depth	None, 10, 20, 30	10
max_features	'sqrt', 'log2'	'log2'
min_samples_split	Random integers between 2 and 10	5
min_samples_leaf	Random integers between 1 and 5	4

Table 4: Hyperparameter distributions used in RandomizedSearchCV

Model Performance

- Use best parameters from RandomSearchCV for final model
- Overall accuracy: 73.5%
- OBB score: 74.7% ➡ no strong evidence of overfitting
 - OBB, test accuracy and CV accuracy close
- Performs well for majority class
 - Recall: 84%
 - Precision: 77%
- Performs worse on majority classes
 - Recall: 54%
 - Precision: 64%
- Result reflect class imbalance, introducing a bias
- Most important predictors: remoteness, longi- and latitude
- Causal interpretaion harder

Hyperparameter	Values Tested	Best Values
n_estimators	[90, 120, 150]	90
max_depth	[10, 20, 30]	10
max_features	['log2']	'log2'
min_samples_split	[4, 5, 6]	4
min_samples_leaf	[3, 4, 5]	3

Table 5: Refined Hyperparameter Grid for GridSearchCV (Random Forest)

Metric / Feature	Overall	Catastrophic (Class 1)	Other (Class 0)
Test Accuracy	0.735		
OOB Score (Train Accuracy)	0.747		
Reduced Train Accuracy	0.727		
Mean CV Accuracy (5-fold)	0.725		
Recall		0.54	0.84
Precision		0.64	0.77
F1 Score		0.58	0.81
Top 3 Predictors	Importance Score		
Remoteness	0.217		
Longitude	0.183		
Latitude	0.102		

Table 6: Summary of Random Forest Results (Catastrophic = 1)

Gradient Boosting

3.3

RandomizedSearchSV and GridSeachSV

Parameter	Values Tested	Best Values
n_estimators	Randomized integers between 100 and 300	112
max_depth	[4 , 5 , 6 , 8 , 10]	4
learning_rate	[0.05 , 0.1]	0.10
gamma	uniform (4.5 , 3)	6.19
colsample_bytree	[0.5 , 0.75]	0.75
min_child_weight	[4 , 5 , 6]	6
scale_pos_weight	[1.8 , 2.0 , 2.2]	2.2

Table 7: Hyperparameter distributions used in RandomizedSearchCV



Parameter	Values Tested	Best Values
n_estimators	112	112
max_depth	[3 , 4 , 5]	3
learning_rate	[0.08, 0.10 , 0.12]	0.08
gamma	[5.89 , 6.19 , 6.49]	5.89
colsample_bytree	0.75	0.75
min_child_weight	6	6
scale_pos_weight	2.2	2.2

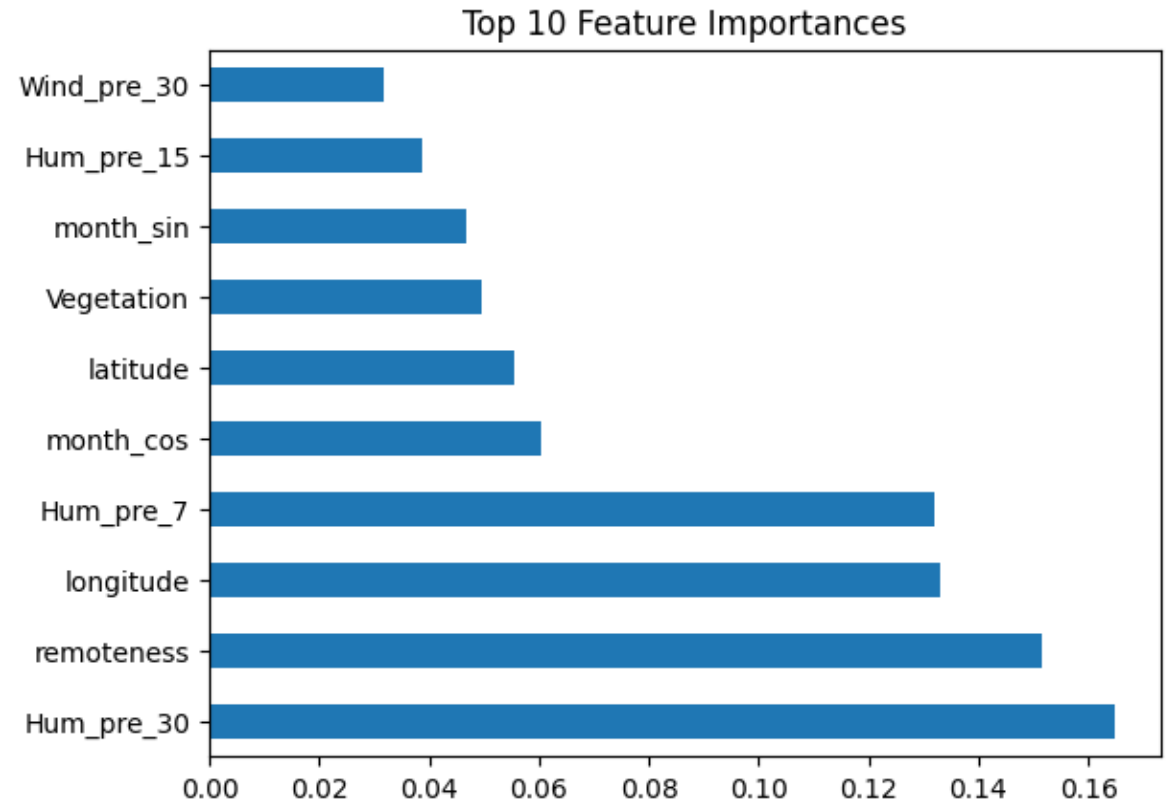
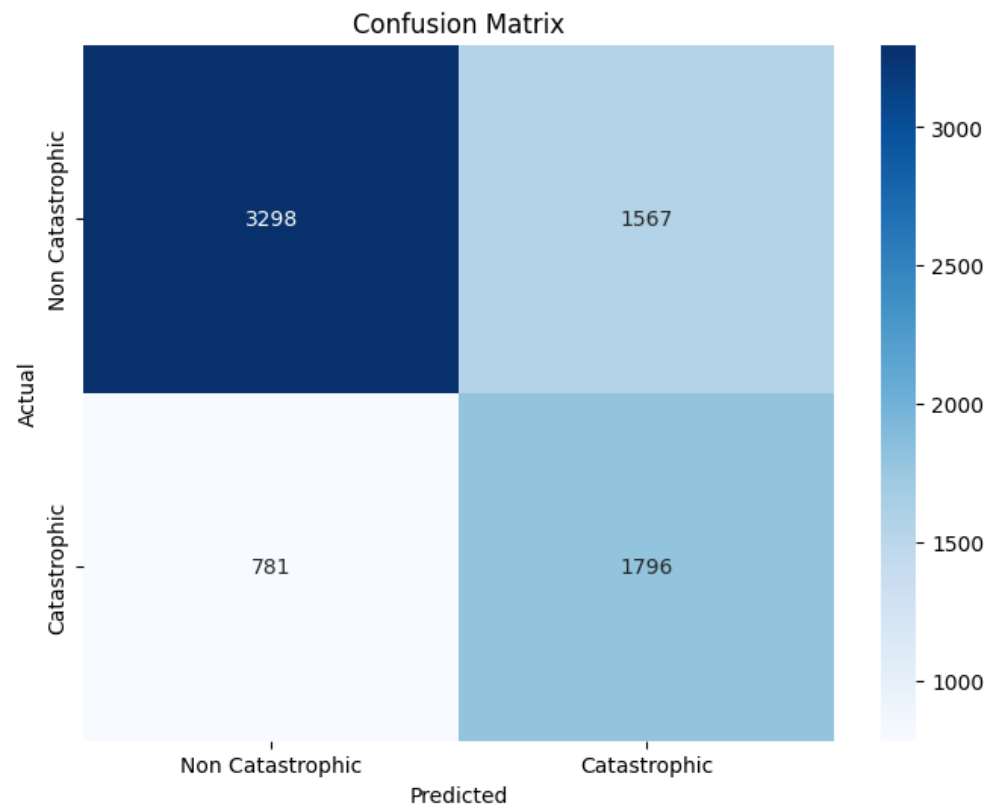
Table 8: Optimal Hyperparameters after GridSeachSV

Final Model

Metric / Feature	Overall	Catastrophic (Class 1)	Other (Class 0)
Test Accuracy	0.68		
Reduced Train Accuracy	0.69		
Mean CV Accuracy (5-fold)	0.67		
Recall		0.70	0.68
Precision		0.53	0.81
F1 Score		0.60	0.74

Table 9: Summary of Model Evaluation (Catastrophic = 1)

Final Model



Neural Network

3.4

Network Design

Type	Description	
Input	Input Features	24
Fully Connected (fc1)	Linear Layer	$in = 24, out = 64$
Activation	ReLU	-
Fully Connected (fc2)	Linear Layer	$in = 64, out = 32$
BatchNorm	BatchNorm1d	-
Activation	ReLU	-
Dropout	Dropout Layer	$p = 0.5$
Fully Connected (fc3)	Linear Layer	$in = 32, out = 1$
Output	Raw Logits	1

Table 10: Network Design.

LR Scheduler

- Focus on **Positive Recall**
- If no increase over 3 epochs -> learning rate x 0.1

Training Process:

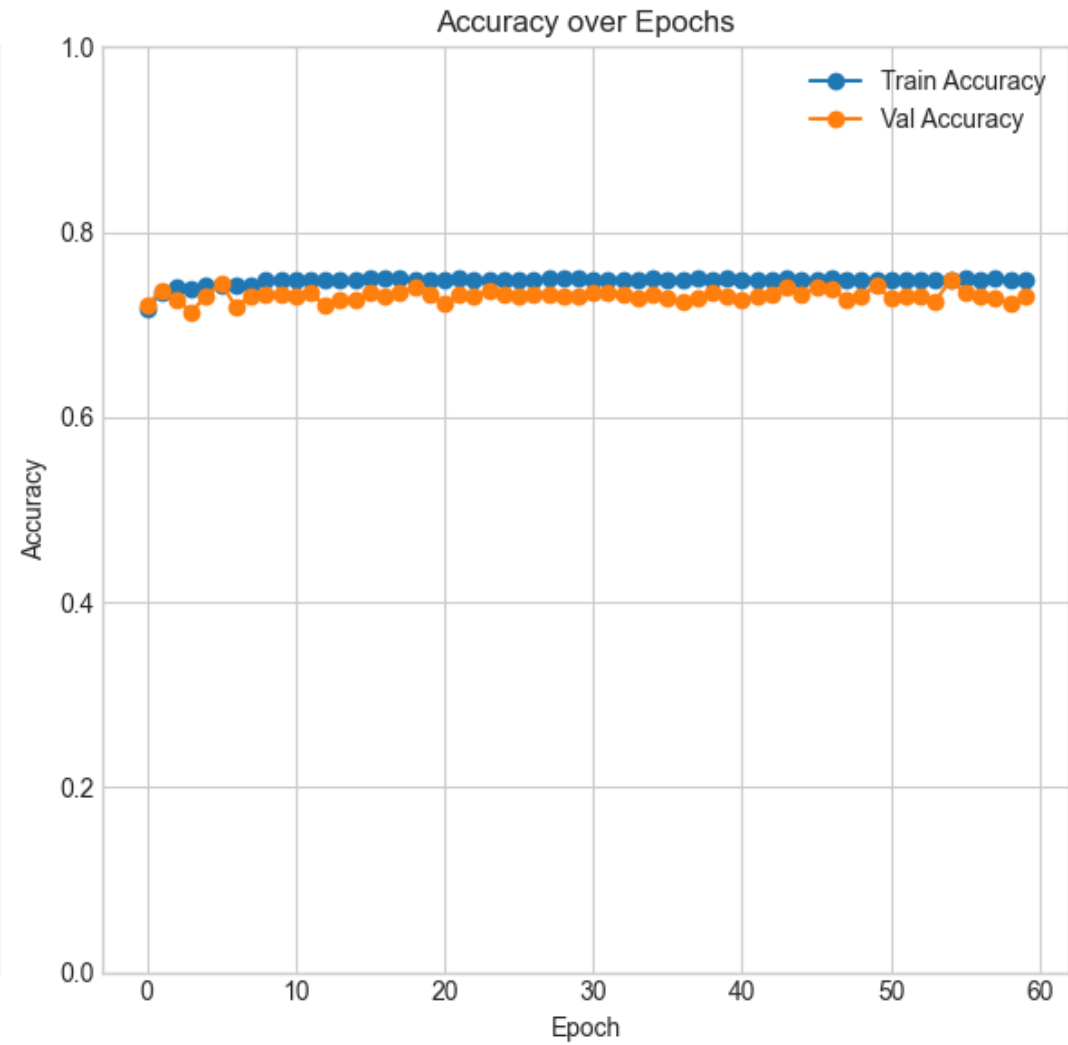
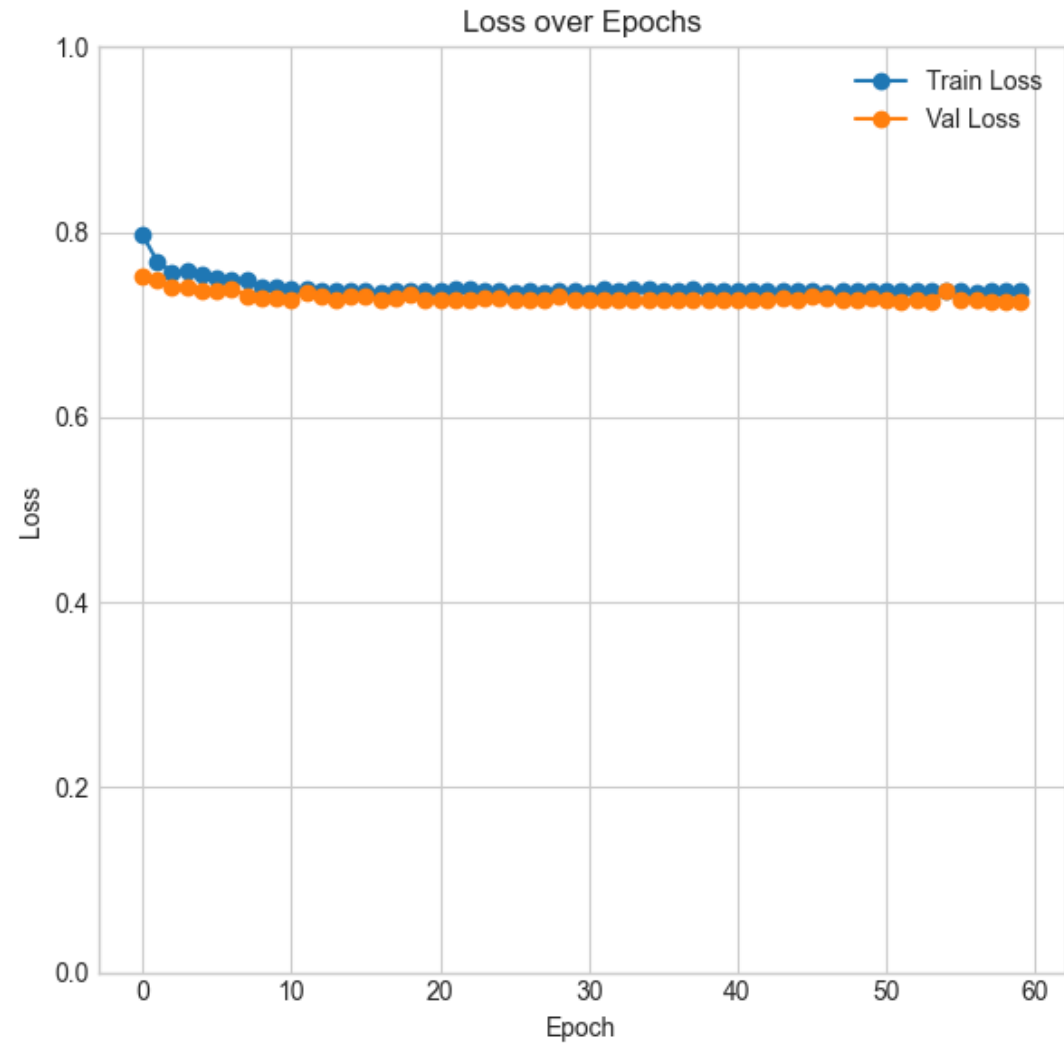
- 60 Epochs
- Adam Optimizer (learning rate: 0.01, weight decay: 1e-4)

```
input :  $\gamma$  (lr),  $\beta_1, \beta_2$  (betas),  $\theta_0$  (params),  $f(\theta)$  (objective)
         $\lambda$  (weight decay), amsgrad, maximize,  $\epsilon$  (epsilon)
initialize :  $m_0 \leftarrow 0$  ( first moment),  $v_0 \leftarrow 0$  (second moment),  $v_0^{max} \leftarrow 0$ 

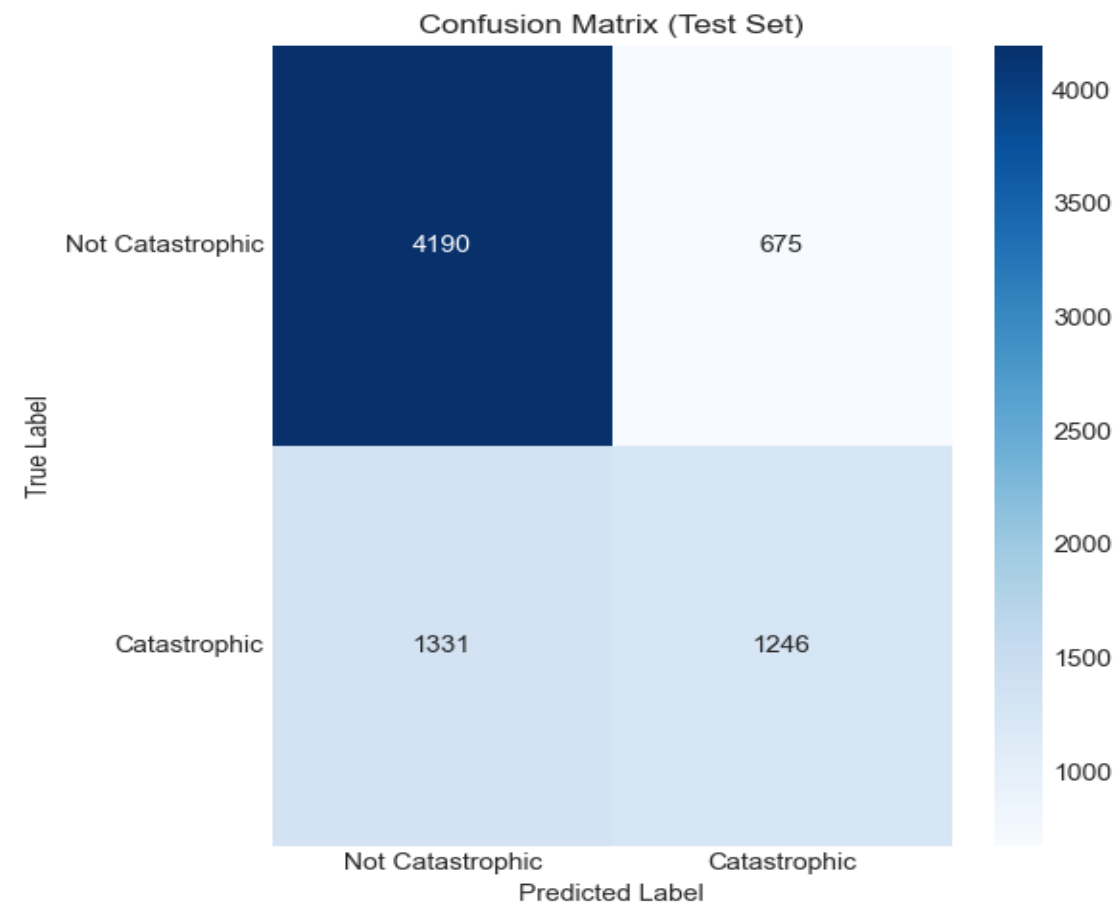
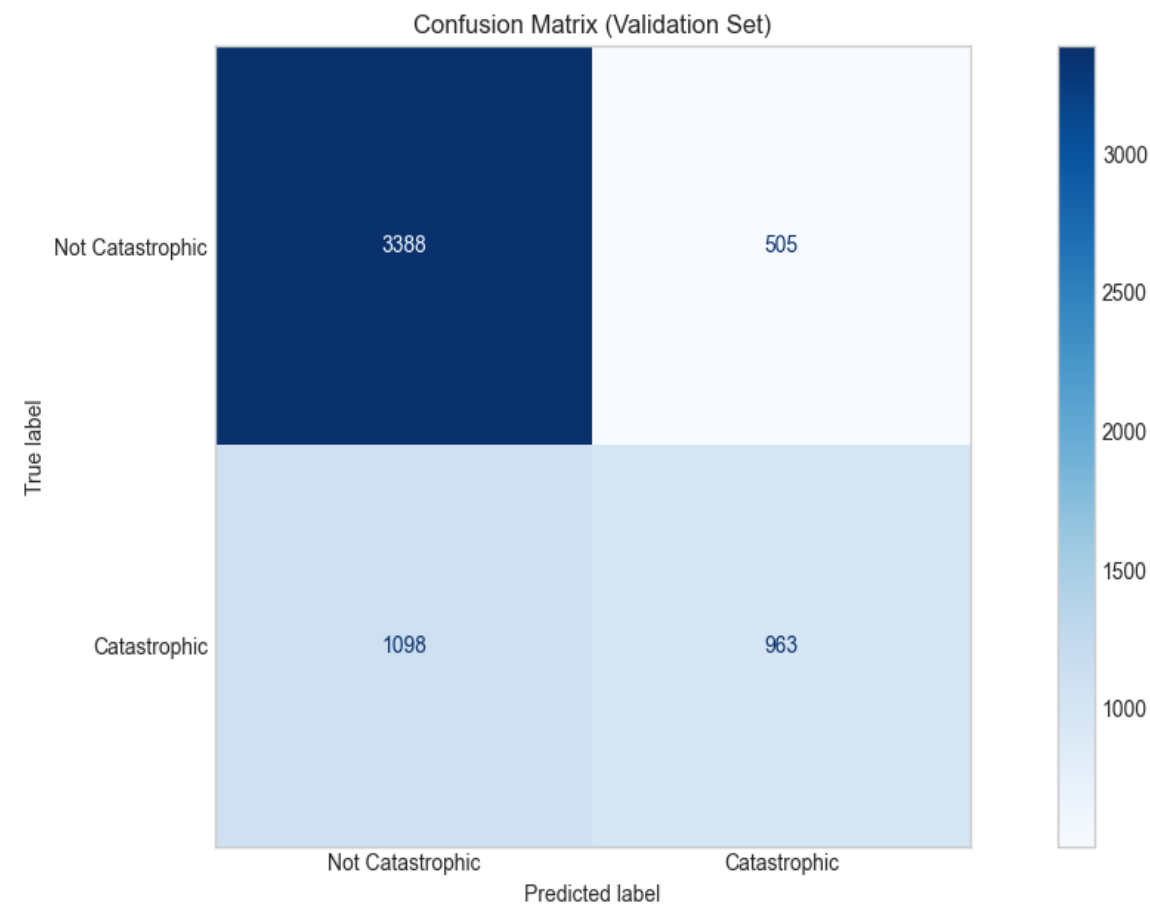
for  $t = 1$  to ... do
    if maximize :
         $g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$ 
    else
         $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ 
    if  $\lambda \neq 0$ 
         $g_t \leftarrow g_t + \lambda \theta_{t-1}$ 
     $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
     $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
     $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
    if amsgrad
         $v_t^{max} \leftarrow \max(v_{t-1}^{max}, v_t)$ 
         $\widehat{v}_t \leftarrow v_t^{max} / (1 - \beta_2^t)$ 
    else
         $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
     $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ 

return  $\theta_t$ 
```

Training



Validation vs. Test Set



Classification Report on Test Set

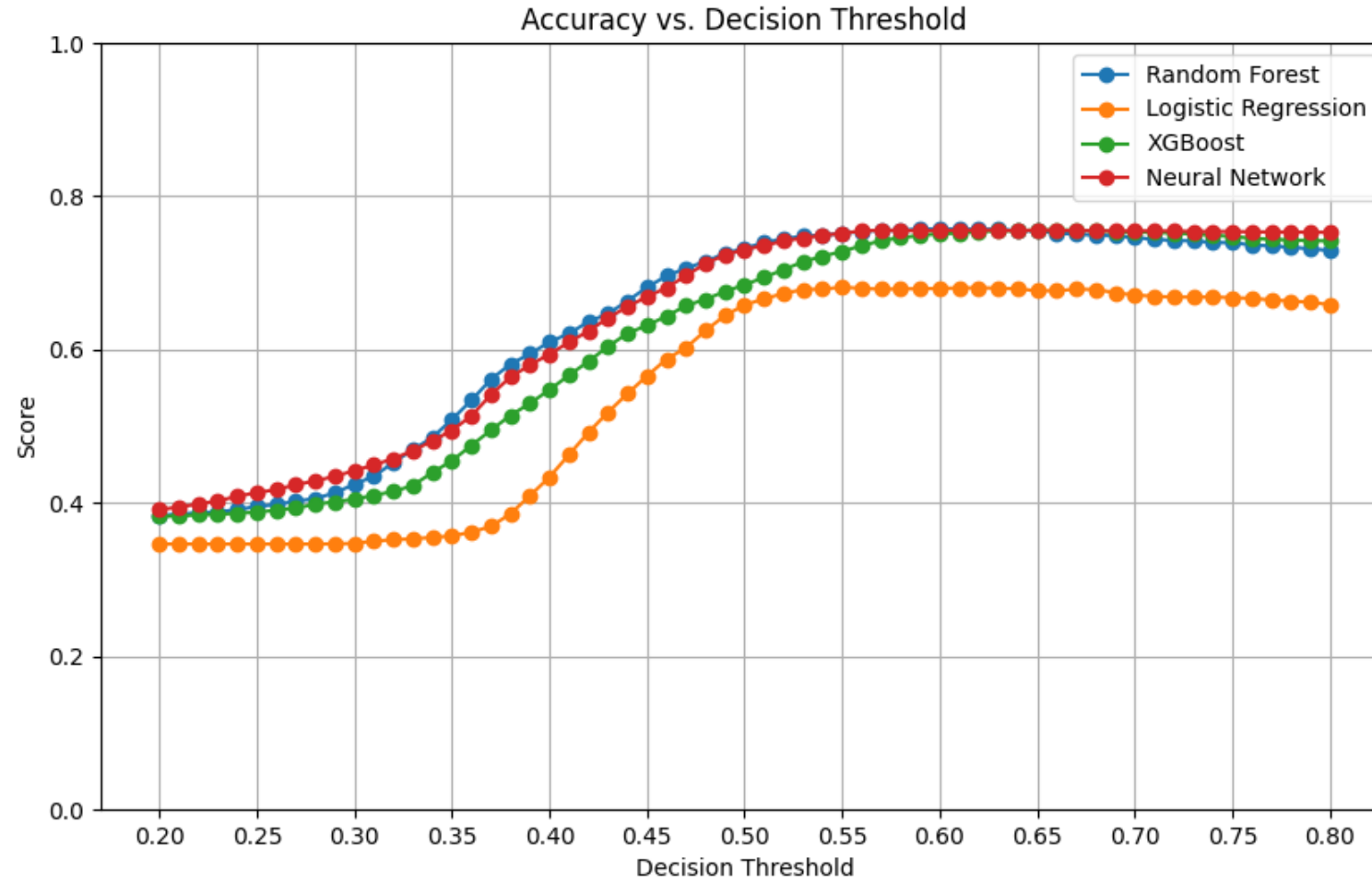
Class	Precision	Recall	F1-score	Support
Not Catastrophic	0.76	0.86	0.81	4865
Catastrophic	0.65	0.48	0.55	2577
Accuracy			0.73	7442
Macro avg	0.70	0.67	0.68	7442
Weighted avg	0.72	0.73	0.72	7442

Table 11: Classification Report on Test Set.

Model Comparison

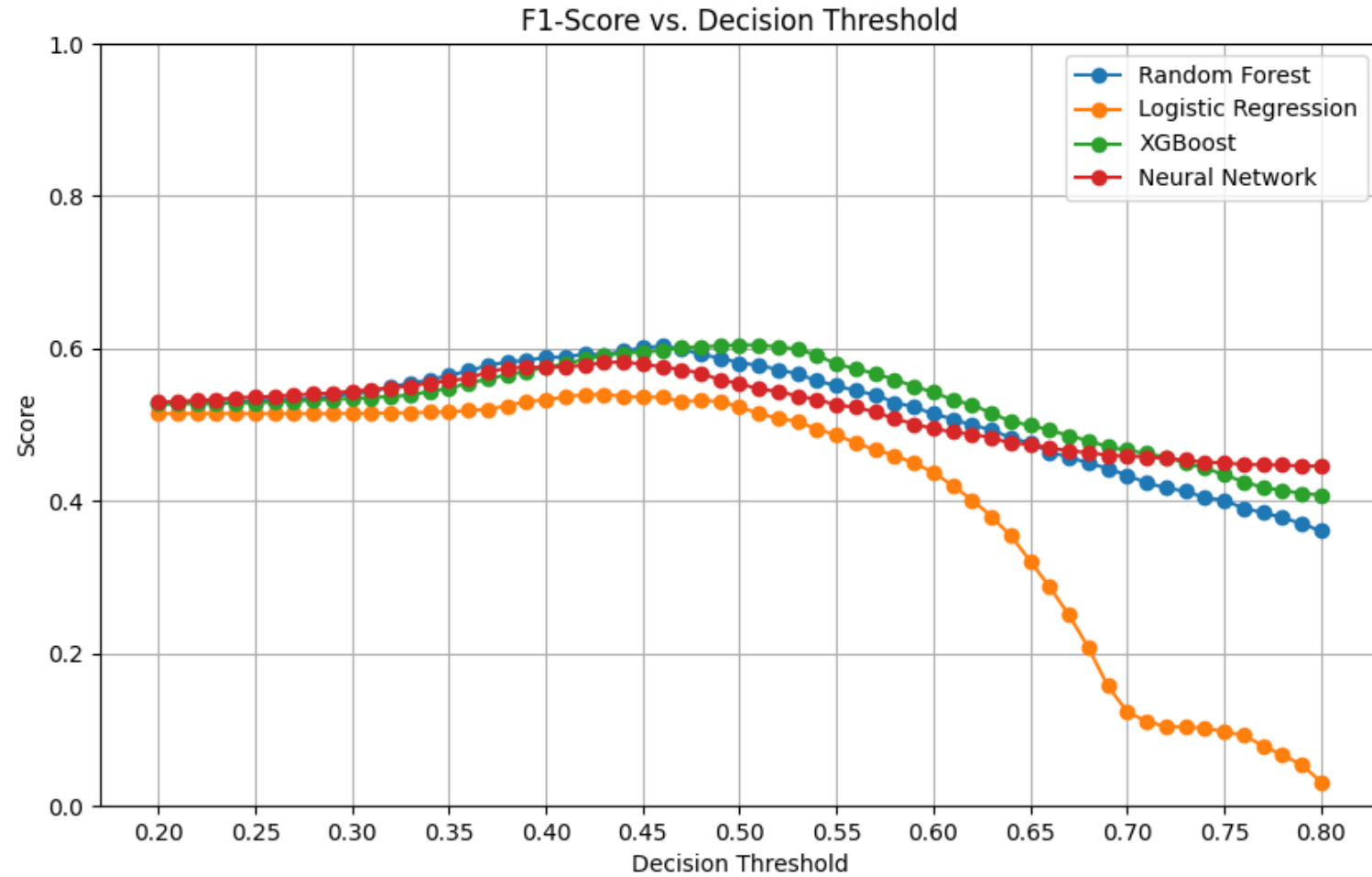
3.5

The Role of the Decision Threshold.

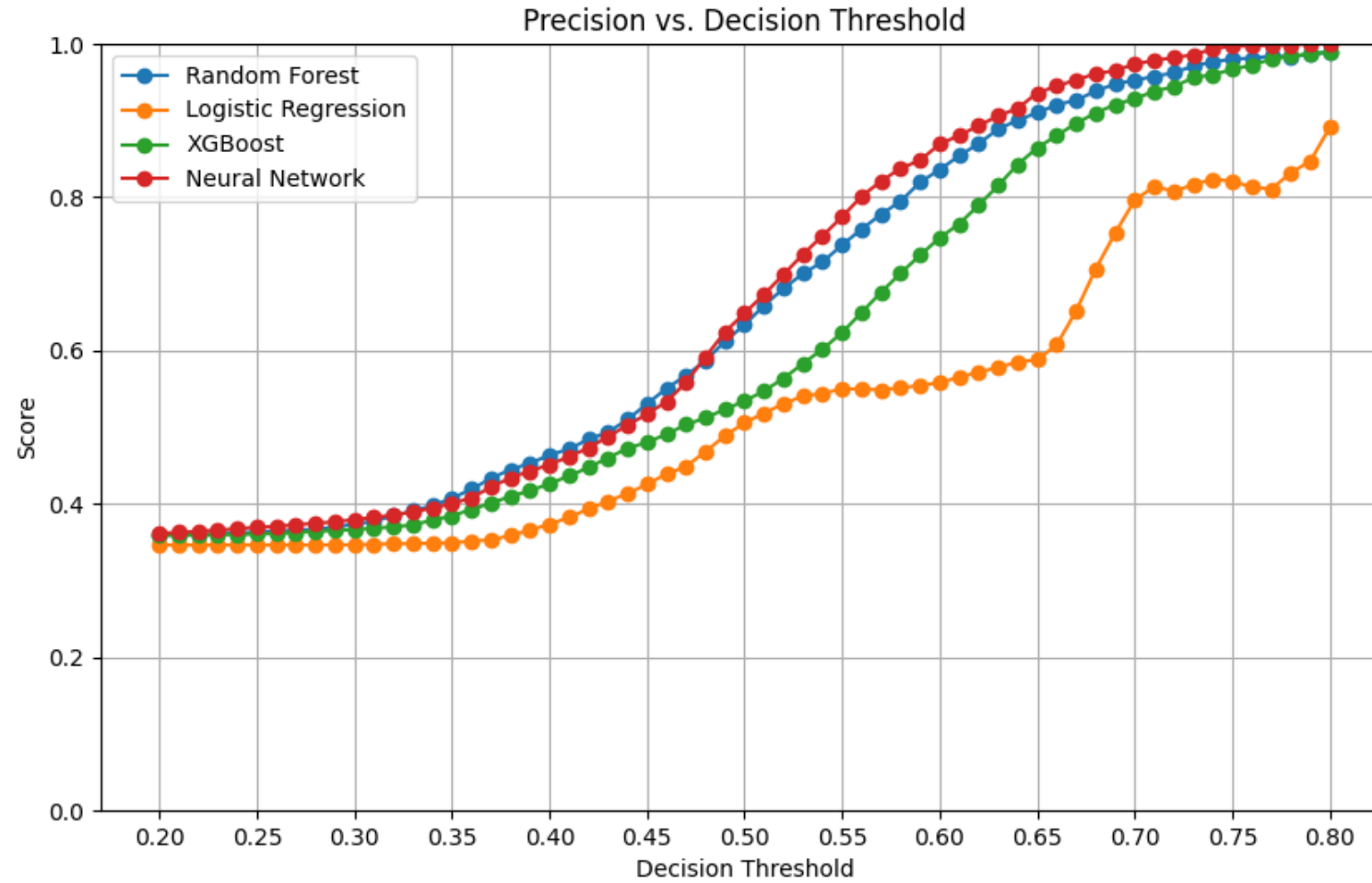


Choosing the best threshold.

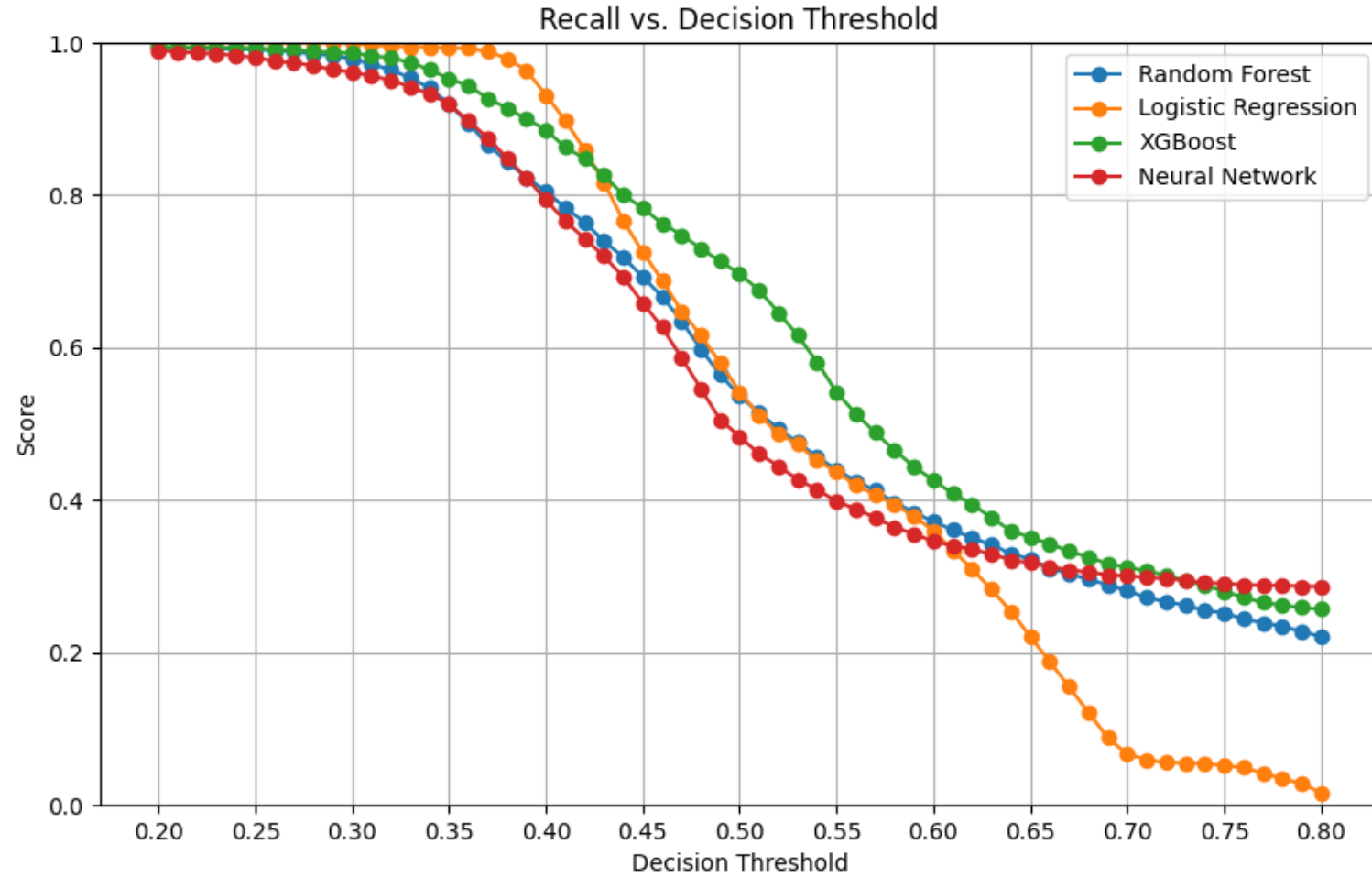
Important: ALWAYS CHOOSE BASED ON TRAINING DATA!



No surprises.



Again, just as expected.



How do the models compare?

Model	Accuracy	Recall	Precision	F1	F1 (weighted)
XGBoost	0.68	0.70	0.53	0.60	0.69
Random Forest with threshold 0.46	0.70	0.67	0.55	0.60	0.70
Ensemble Model (Majority Vote)	0.69	0.66	0.54	0.60	0.70
Neural Network with threshold 0.44	0.66	0.69	0.50	0.58	0.66
Random Forest	0.73	0.54	0.63	0.58	0.73
Logistic Regression with threshold 0.43	0.52	0.82	0.40	0.54	0.51
Logistic Regression	0.66	0.54	0.51	0.52	0.66

Table 12: Test Set Model performance sorted by F1 score.

Conclusion

4

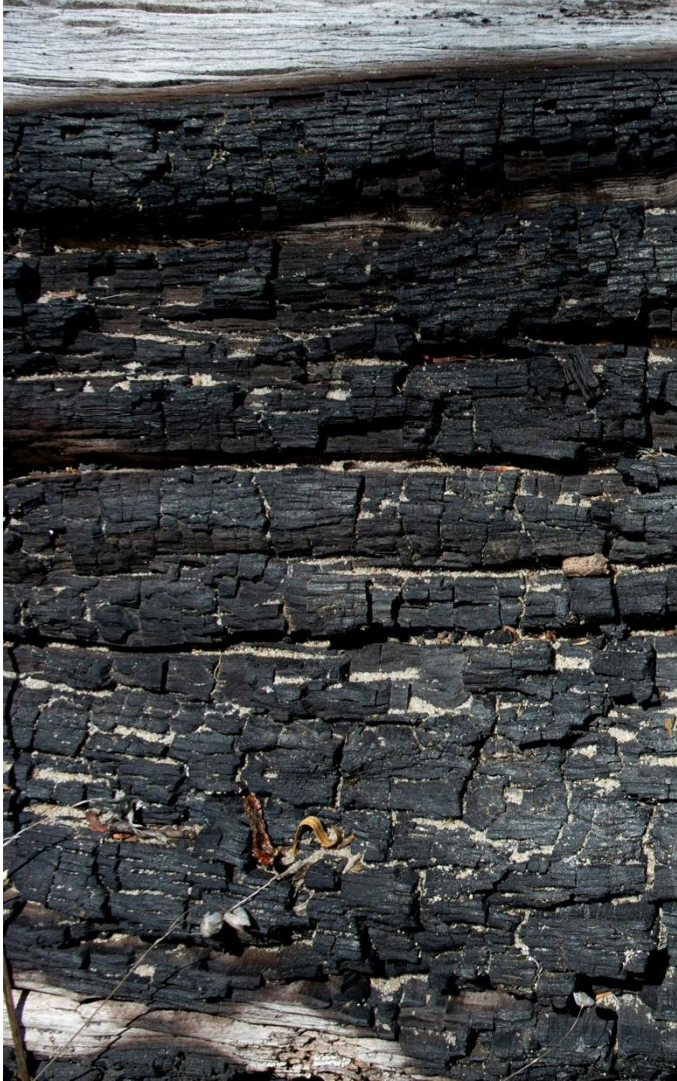
How useful are our models?

Better than guessing? Yes!

But professionals do not guess either.

We lack important information like weather forecasts.
But: We can make better use of the data we have!

How can we improve our model?



Feature Engineering:

- Time to previous fires
- Approximate weather forecasts from weather until containment

More data, better data!

https://unsplash.com/de/fotos/ein-stuck-holz-das-verbrannt-wurde-HFRdT2570_U



Thank you for listening.

Victoria Vorre
Leon Kriesmair
William von Tucher